Описание практической реализации ВКР на тему «Разработка клиентской части веб-приложения для управления экскурсиями и турами»

1. Анализ функциональных требований

1.1 Анализ модуля авторизации и аутентификации

Модуль авторизации и аутентификации является ключевым элементом безопасности приложения. В ходе анализа были выделены следующие функциональные требования: регистрация новых пользователей с указанием персональных данных, вход в систему с проверкой учетных данных, восстановление пароля, а также выход из системы.

Важным требованием является защита маршрутов от несанкционированного доступа и разграничение прав на основе ролей пользователей: туристы (клиенты), гиды (организаторы туров) и администраторы системы.

Дополнительно требуется реализация механизма защиты от множественных неудачных попыток входа с временной блокировкой.

1.2 Анализ модуля каталога туров и экскурсий

Модуль каталога туров выступает центральным элементом приложения. Функционал включает отображение списка доступных туров в виде карточек, содержащих название, краткое описание, даты проведения, время начала, длительность, стоимость, локацию и уровень сложности.

Для каждого тура требуется детальная страница с галереей изображений, полной программой мероприятия, перечнем включенных услуг, требованиями к участникам и расписанием доступных дат для бронирования. Также предусмотрена возможность создания, редактирования и удаления туров для администраторов. Блок-схема, отображающая логику работы главной страницы отображен на рисунке 1.1. Макет страницы представлен на рисунке 1.2. Блок-схема, отображающая логику работы страницы деталей тура отображен на рисунке 1.3. Макет страницы представлен на рисунке 1.4.

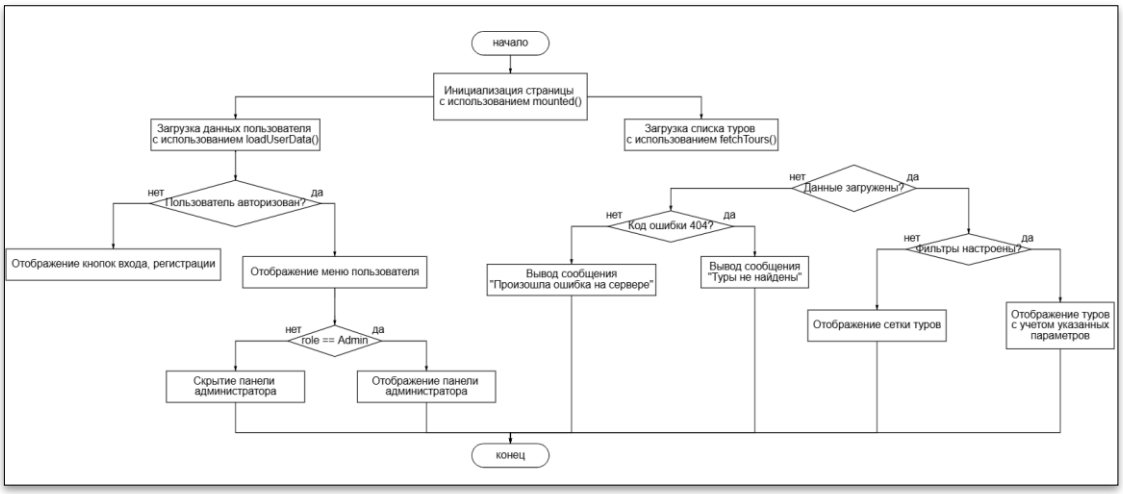

Рисунок 1.1 – Блок-схема логики работы главной страницы

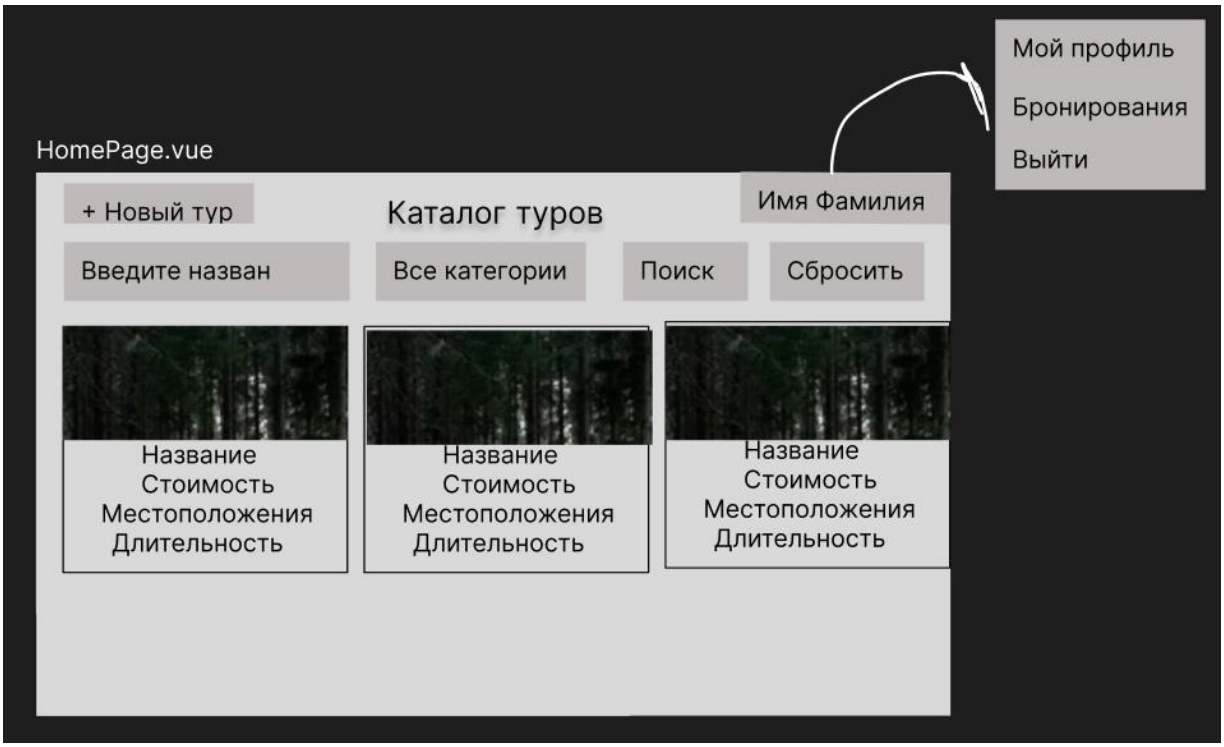

Рисунок 1.2 – Макет приложения HomePage.vue

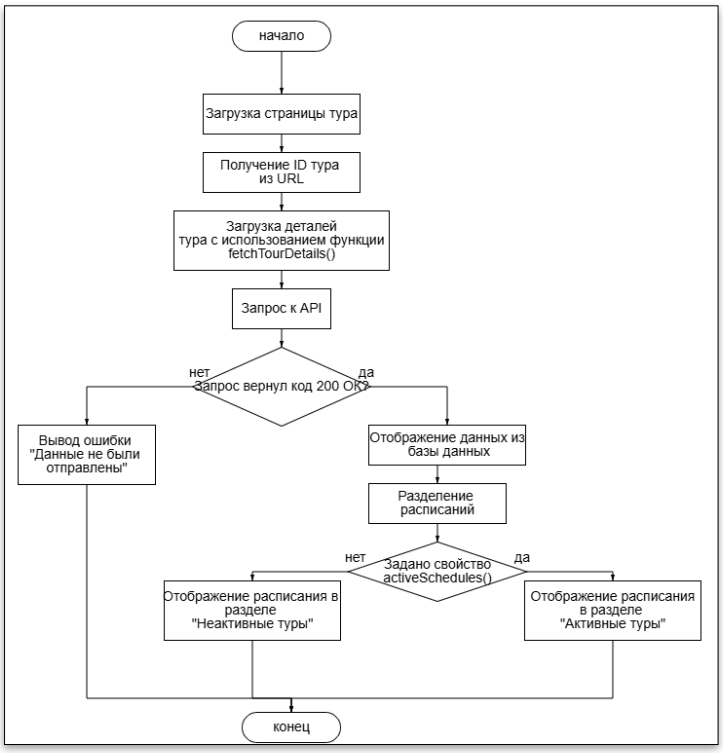

Рисунок 1.3 – Блок-схема логики работы страницы деталей тура

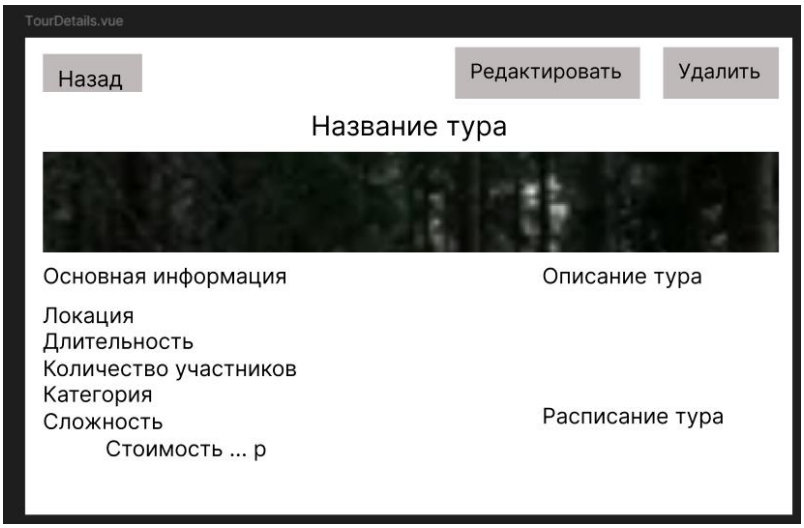

Рисунок 1.4 – Макет приложения ToursDetails.vue

1.3 Анализ модуля поиска и фильтрации

Модуль поиска и фильтрации обеспечивает удобную навигацию по каталогу. Требуется реализовать поиск по названию и описанию туров, а также фильтрацию результатов по типу тура, ценовому диапазону, продолжительности, уровню сложности и доступным датам. Все фильтры должны применяться динамически без перезагрузки страницы, с возможностью сброса настроек фильтрации.

1.4 Анализ модуля управления бронированиями

Модуль управления бронированиями позволяет пользователям создавать новые бронирования на доступные даты, просматривать историю своих бронирований с указанием уникального номера, деталей и статуса, а также отменять или изменять существующие бронирования. Для администраторов требуется возможность просмотра всех бронирований системы, изменения их статусов и фильтрации по пользователям. Статусы бронирований включают: ожидание, подтверждено, завершено, отменено.

Блок-схема, отображающая логику работы главной страницы отображен на рисунке 1.6. Макет страницы представлен на рисунке 1.5.

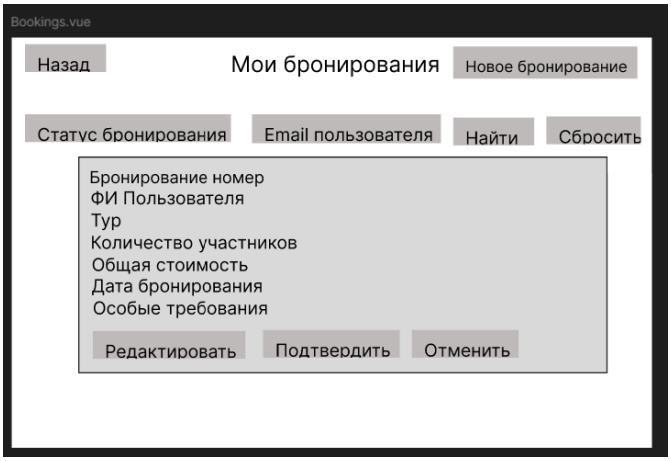

Рисунок 1.5 – Макет приложения Bookings.vue

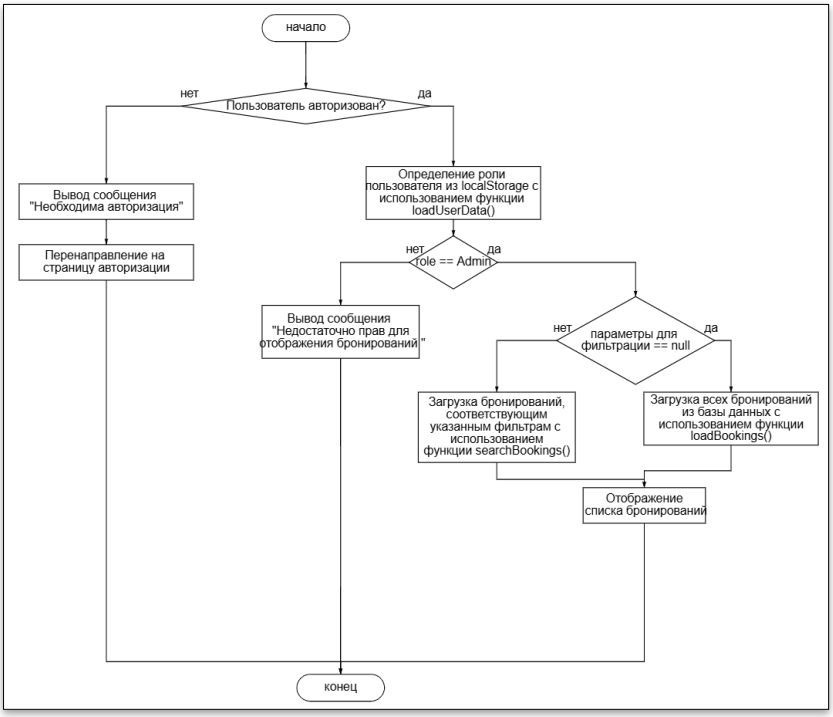

Рисунок 1.1 – Блок-схема логики работы функции загрузки бронирований

1.5 Анализ личного кабинета пользователя

Личный кабинет пользователя предоставляет авторизованным пользователям возможность редактирования личной информации: имени, фамилии, контактных данных. Также в личном кабинете реализована функция изменения пароля с проверкой текущего пароля и валидацией сложности нового. Для администраторов в личном кабинете доступна дополнительная информация о роли и правах в системе.

Блок-схема, отображающая логику работы главной страницы отображен на рисунке 1.7. Макет страницы представлен на рисунке 1.8.

Рисунок 1.7 – Блок-схема логики работы функции просмотра профиля

Рисунок 1.8 – Макет приложения Profile.vue

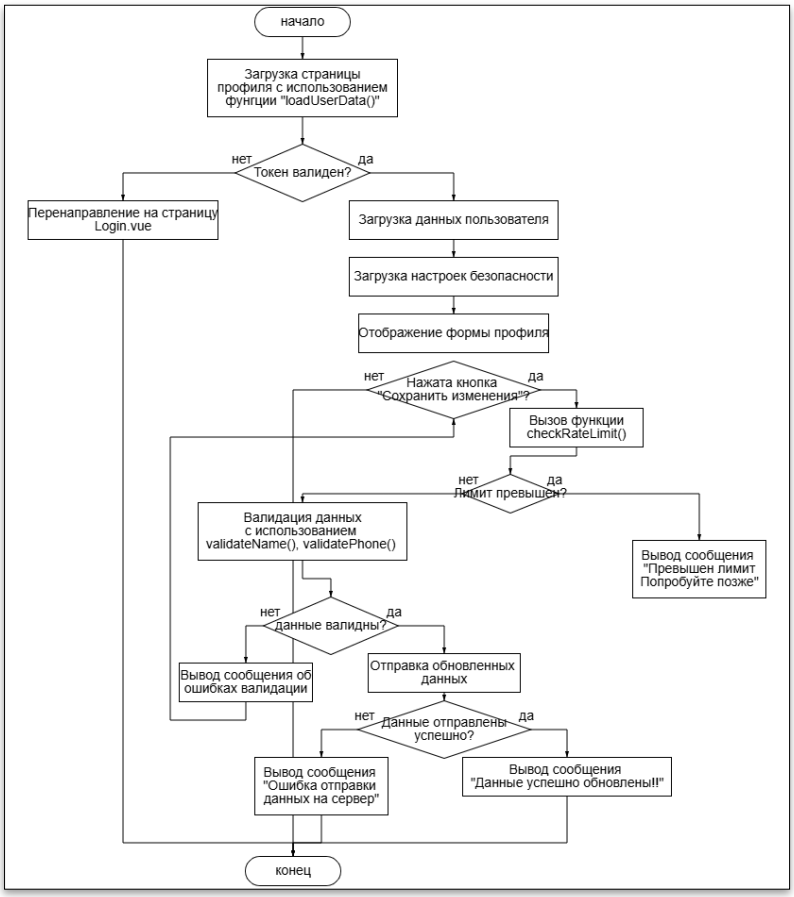

Рисунок 1.**9** – Блок-схема логики работы функции редактирования профиля

1.6 Разработка маршрутов навигации между страницами приложения (Рисунок 1.10)

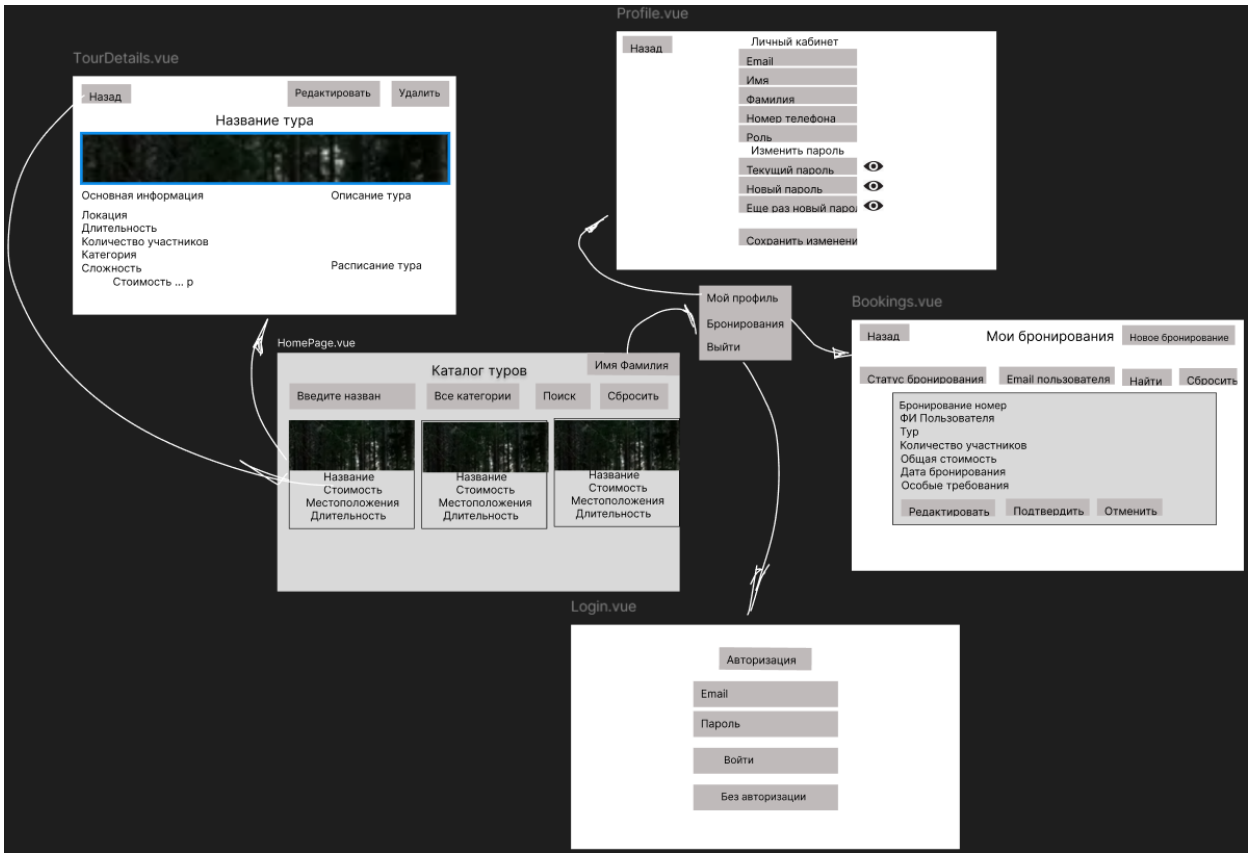

Рисунок 1.10 – Маршруты навигации между страницами приложения

2. Выбор архитектуры и средств разработки

2.1 Обоснование выбора фреймворка Vue 3

В качестве основного фреймворка был выбран Vue 3 с Composition API. Данный выбор обусловлен следующими преимуществами:

– Composition API позволяет группировать логику по функциональности, а не по типам методов, что обеспечивает гибкость организации кода;

– Composition API разработан с учетом TypeScript, что обеспечивает превосходную типизацию и автодополнение;

– Vue 3 имеет полностью переработанную реактивную систему, обеспечивающую высокую производительность; фреймворк позволяет исключать неиспользуемый код при сборке.

2.2 Обоснование выбора TypeScript

TypeScript выбран как язык разработки, поскольку обеспечивает статическую типизацию, выявляющую ошибки на этапе компиляции, а не во время выполнения. Типы служат формой документации, упрощая поддержку кода, а интерфейсы четко описывают ожидаемую структуру данных. TypeScript обеспечивает автодополнение, навигацию по коду и рефакторинг в современных IDE, что значительно ускоряет разработку. Ошибки, такие как обращение к несуществующим свойствам объектов или неправильные типы параметров, обнаруживаются на этапе разработки.

2.3 Выбор инструментов для маршрутизации

Для маршрутизации применяется Vue Router, обеспечивающий нативную интеграцию с Vue, бесшовную совместимость и использование реактивности для навигационного состояния. Библиотека предоставляет полную типизацию для маршрутов, параметров и сторожевых хуков, что предотвращает ошибки в путях. Поддерживается динамическая загрузка компонентов для разделения кода, что критически важно для производительности крупных SPA-приложений. Система вложенных маршрутов позволяет создавать сложные интерфейсы с независимыми разделами, а навигационные хуки обеспечивают гибкую защиту маршрутов

2.4 Выбор библиотеки для управления состоянием

Для управления состоянием приложения выбрана Pinia – официальная библиотека управления состоянием для Vue, которая пришла на смену Vuex. Pinia обеспечивает полную интеграцию с TypeScript, интуитивно понятный API и модульную структуру хранилища. Библиотека поддерживает реактивность Vue и предоставляет удобные инструменты для организации глобального состояния приложения.

2.5 Выбор HTTP-клиента

Для отправки HTTPS-запросов выбран встроенный Fetch API, что обеспечивает нативную интеграцию с браузером без дополнительных зависимостей, повышая производительность загрузки. Fetch API является официальным стандартом для работы с сетевыми запросами в современных браузерах, предоставляет удобную работу с асинхронными операциями, автоматическую обработку данных и детальный контроль над HTTP-запросами и ответами

2.6 Выбор библиотеки UI-компонентов

Для ускорения разработки и обеспечения единообразного внешнего вида выбрана библиотека Ant Design, предоставляющая богатый набор готовых, хорошо протестированных и доступных компонентов. Библиотека включает все необходимые элементы интерфейса: кнопки, модальные окна, формы, таблицы, селекты и другие компоненты, что позволяет сосредоточиться на бизнес-логике приложения.

2.7 Выбор инструментов для работы с формами

Для работы с формами используется встроенный реактивный подход Vue 3 в сочетании с кастомными хуками валидации. Реализована система валидации с визуальной обратной связью, проверкой обязательных полей, формата email, минимальной длины пароля и соответствия числовых значений допустимым диапазонам.

2.8 Выбор библиотеки для кэширования данных (TanStack Query)

Для кэширования данных и управления серверным состоянием используется TanStack Query (ранее Vue Query). Библиотека автоматически кэширует результаты запросов к API, выполняет фоновое обновление данных, предоставляет удобные инструменты для управления состоянием загрузки и ошибок, а также поддерживает пагинацию, что необходимо для эффективной работы с большими списками.

3 Описание архитектуры клиентского приложения

3.1 Организация структуры проекта

Для кэширования данных и управления серверным состоянием используется TanStack Query (ранее Vue Query). Библиотека автоматически кэширует результаты запросов к API, выполняет фоновое обновление данных, предоставляет удобные инструменты для управления состоянием загрузки и ошибок, а также поддерживает пагинацию, что необходимо для эффективной работы с большими списками.

3.2 Компонентный подход к построению интерфейса

Интерфейс приложения декомпозирован на множество мелких, переиспользуемых компонентов. Каждый компонент отвечает за отдельную часть интерфейса и может быть легко повторно использован в различных частях приложения.

Компонент карточки тура TourCard принимает через пропсы все необходимые данные: идентификатор, название, описание, цену, продолжительность, уровень сложности и URL изображения. Компонент полностью типизирован с использованием TypeScript, включает логику обрезки длинного описания и использует атрибут loading="lazy" для отложенной загрузки изображений.

3.3 Реализация маршрутизации с защитой по ролям

Для обеспечения разделения прав доступа реализована маршрутизация с проверкой авторизации и роли пользователя с помощью навигационных хуков Vue Router. Защищенные маршруты проверяют наличие JWT-токена и соответствие роли пользователя требуемым правам доступа. В конфигурации маршрутов выделены публичные маршруты (доступны всем), защищенные маршруты (только для авторизованных пользователей) и административные маршруты (только для администраторов).

3.4 Организация глобального состояния приложения

Для управления глобальным состоянием приложения используются хранилища Pinia, организованные по модульному принципу. Хранилище authStore отвечает за состояние аутентификации, хранит информацию о текущем пользователе, статусе авторизации и предоставляет действия для входа, регистрации и выхода из системы. Хранилище bookingStore управляет состоянием бронирований, включая фильтрацию и поиск.

3.5 Типизация данных с использованием TypeScript

Все модели данных в приложении строго типизированы с использованием TypeScript интерфейсов. Определены типы для пользователей, туров, расписаний, бронирований и ответов API. Это обеспечивает надежность кода, предотвращает ошибки на этапе разработки и служит документацией для структуры данных.

4 Взаимодействие с серверной частью

4.1 Настройка HTTP-клиента и интерцепторов

Для взаимодействия с серверной частью используется Fetch API с настроенными обертками для обработки запросов. Реализованы функции для автоматического добавления JWT-токена в заголовки запросов, централизованной обработки ошибок и управления состояниями загрузки. При получении ошибки 401 (Unauthorized) выполняется автоматический выход пользователя из системы.

4.2 Реализация сервисного слоя для работы с API

Для каждой сущности создан отдельный сервис, инкапсулирующий логику взаимодействия с API. Сервис authService содержит методы для входа, регистрации и выхода из системы. Сервис tourService предоставляет методы для получения списка туров, детальной информации, создания, обновления и удаления туров. Сервис bookingService включает методы для работы с бронированиями и расписаниями.

4.3 Обработка ответов сервера и ошибок

Обработка ошибок реализована на глобальном уровне с дифференциацией типов ошибок: ошибки аутентификации, ошибки валидации, ошибки сервера и сетевые ошибки. Для каждого типа ошибок предусмотрено соответствующее уведомление пользователя с понятным сообщением и рекомендациями по действиям.

4.4 Интеграция с TanStack Query для кэширования данных

Для оптимизации производительности используется TanStack Query, автоматически кэширующий результаты запросов к API. Настроены параметры кэширования: время жизни кэша (5 минут), автоматическое фоновое обновление и инвалидация кэша при выполнении мутаций. Это значительно уменьшает количество запросов к серверу и ускоряет отображение данных

4.5 Модели данных и TypeScript интерфейсы

Определены следующие основные интерфейсы данных:

– User – пользователь системы (id, email, имя, фамилия, телефон, роль);

– Tour – туристический тур (id, название, описание, длительность, стоимость, локация, сложность, изображения);

– TourSchedule – расписание тура (id, дата начала, дата окончания, доступно мест, гид);

– Booking – бронирование (id, номер, тур, дата, участники, стоимость, статус, требования).

5 Реализация ключевых функциональных модулей

5.1 Реализация модуля авторизации и регистрации

Модуль авторизации реализован на странице Login.vue с формой ввода email и пароля. Реализована клиентская валидация с проверкой формата email и минимальной длины пароля. При отправке формы кнопка блокируется и отображается индикатор загрузки. После 5 неудачных попыток входа включается временная блокировка на 15 минут. При успешной авторизации JWT-токен сохраняется в localStorage, а пользователь перенаправляется на главную страницу.

Интерфейс страницы profile.vue представлен на рисунке 5.1.

Реализован гостевой режим через кнопку «Продолжить без авторизации», предоставляющий ограниченный функционал просмотра каталога без возможности бронирования.

Интерфейс страницы Login.vue представлен на рисунке 5.1.

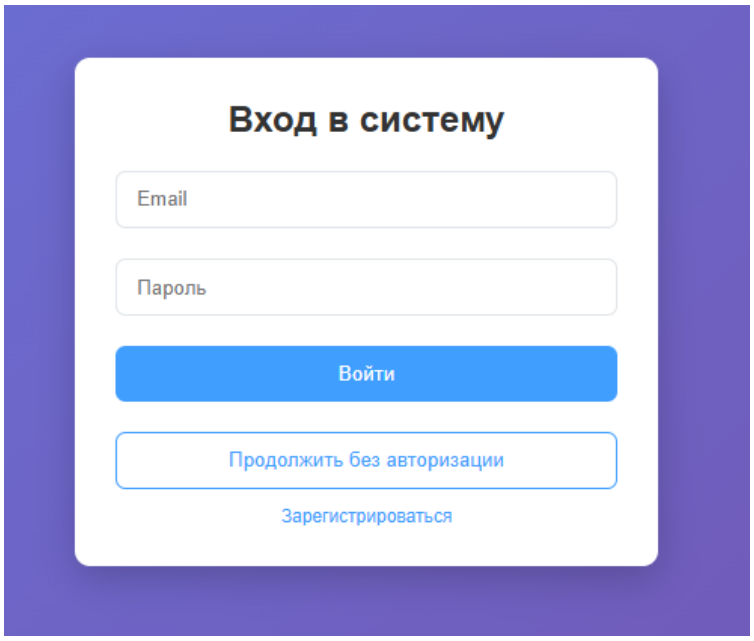

Рисунок 5.1 – Страница авторизации пользователя Login.vue

5.2 Реализация модуля каталога туров

Главная страница HomePage.vue реализует отображение каталога туров в виде сетки карточек. При монтировании компонента выполняется загрузка туров через API с отображением состояния загрузки. Для администраторов отображается панель с кнопкой создания нового тура, открывающей модальное окно с формой для ввода всех параметров тура.

Интерфейс страницы HomePage.vue представлен на рисунке 5.1.

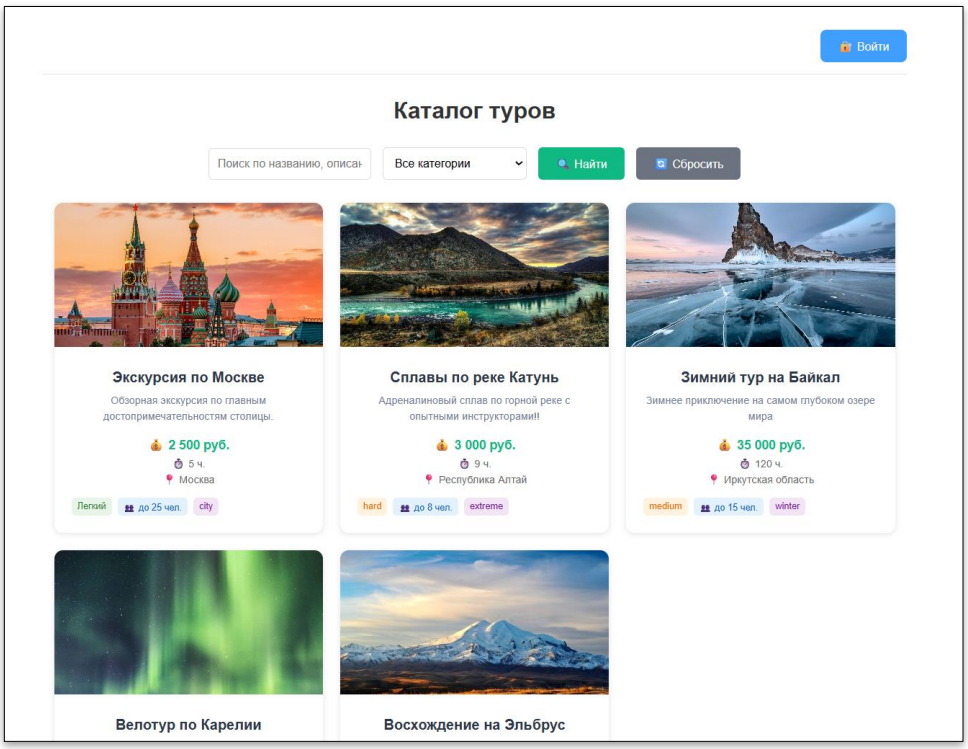

Рисунок 5.2 – Главная страница HomePage.vue

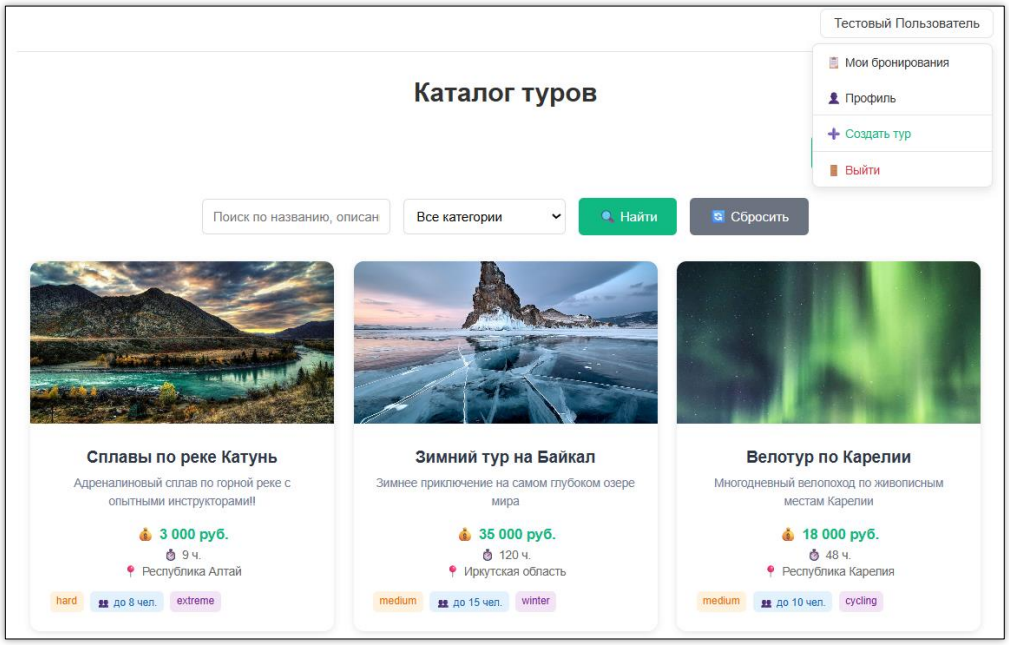

Рисунок 5.3 – Интерфейс страницы HomePage.vue авторизованного пользователя

5.3 Реализация модуля детального просмотра тура

Страница TourDetails.vue загружает полную информацию о туре по ID из параметров маршрута. Отображается галерея изображений, подробное описание, характеристики, программа тура и список включенных услуг. Блок расписания показывает доступные даты проведения с указанием количества свободных мест. Для администраторов доступны кнопки редактирования и удаления тура.

Интерфейс страницы TourDetails.vue представлен на рисунке 5.4.

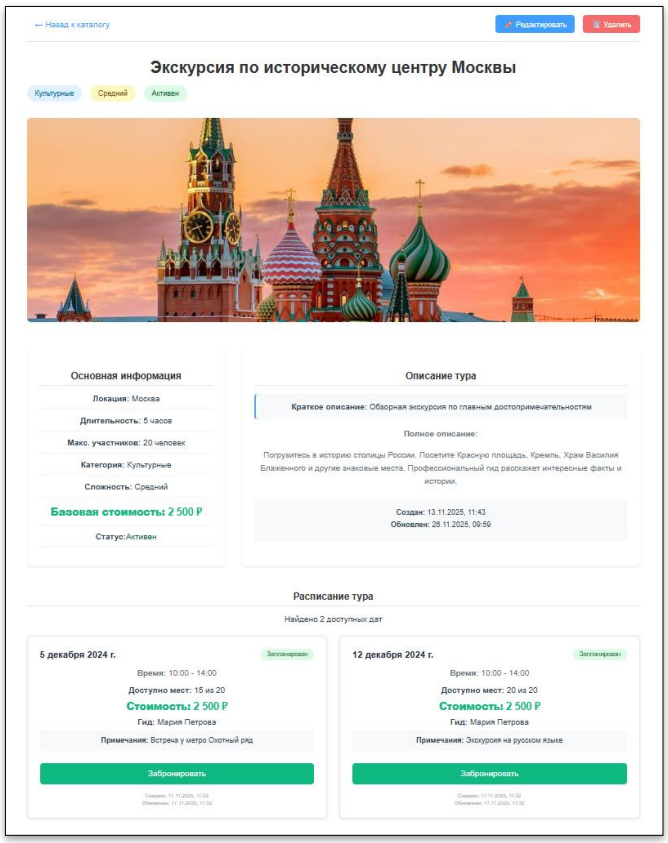

Рисунок 5.4 – Страница TourDetails.vue

5.4 Реализация системы поиска и фильтрации

На главной странице реализована панель фильтрации с полем текстового поиска и выпадающим списком для выбора категории. При изменении параметров фильтрации автоматически выполняется новый запрос к API с обновленными параметрами. Реализована кнопка сброса всех фильтров для возврата к полному каталогу.

Интерфейс страницы HomePage.vue представлен на рисунке 5.5.

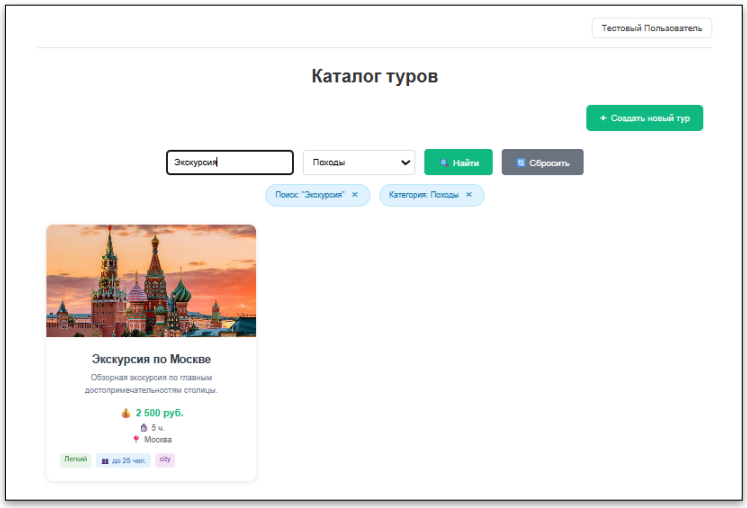

Рисунок 5.5 – Система фильтрации страницы HomePage.vue

Были задействованы следующие маршруты:

– «GET /api/tours» для получения всех туров;
– «GET /api/tours/search?q={query}» для поиска туров;
– «GET /api/tours/category/{category}» для фильтрации по категориям;
– «POST /api/tours» для создания нового тура.

5.5 Реализация модуля бронирования туров

Страница Bookings.vue отображает список бронирований в карточном формате. Для администраторов доступна фильтрация по статусам и поиск по email пользователя. Реализовано модальное окно создания нового бронирования с выбором доступного расписания, динамическим расчетом стоимости и контролем доступных мест.

Для каждого бронирования доступны действия: подтверждение, завершение, отмена (с указанием причины) и редактирование количества участников и особых требований. При изменении статуса или отмене бронирования запрашивается подтверждение действия.

Интерфейс страницы Bookings.vue представлен на рисунке 5.6.

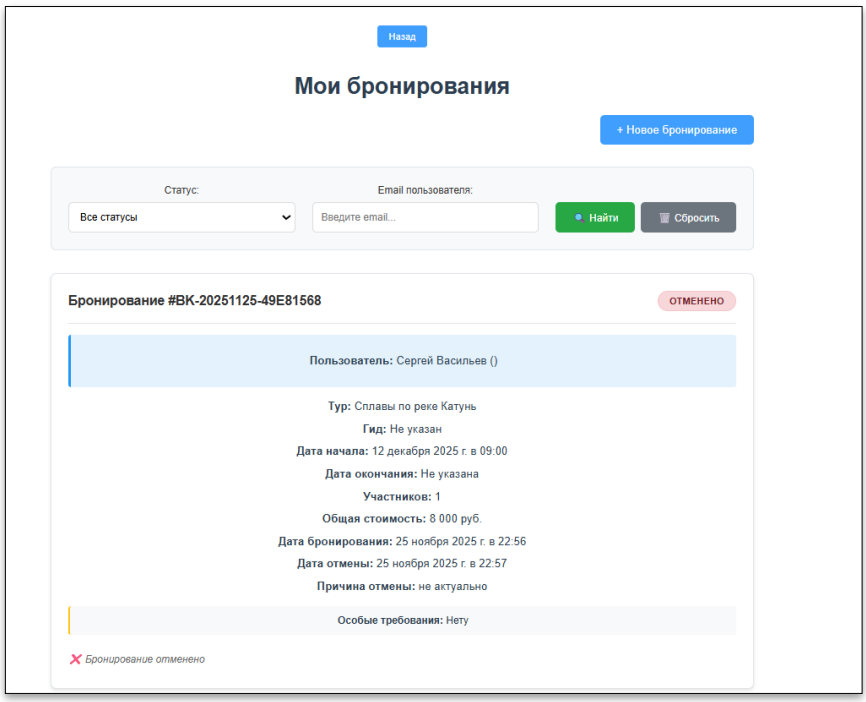

Рисунок 5.6 – Страница Bookings.vue

5.6 Реализация личного кабинета пользователя

Страница Profile.vue отображает информацию о пользователе с возможностью редактирования. Реализована форма изменения персональных данных с валидацией в реальном времени. Отдельная форма для смены пароля включает проверку текущего пароля, визуальную индикацию сложности нового пароля и контроль совпадения при подтверждении.

Интерфейс страницы profile.vue представлен на рисунке 5.7.

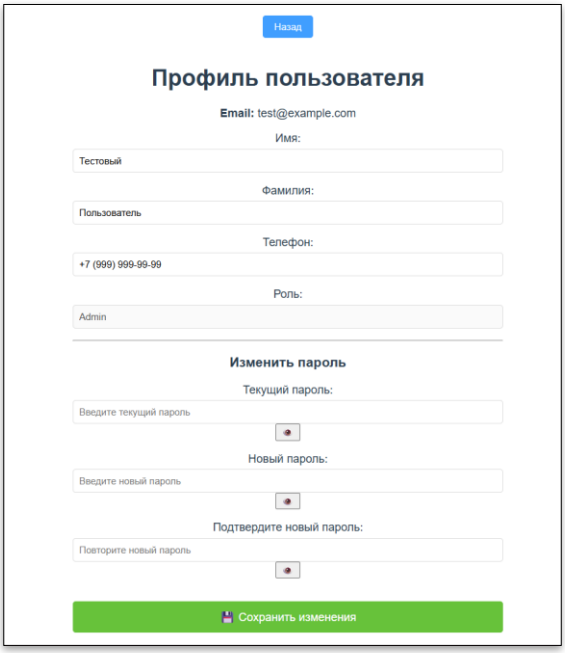

Рисунок 5.7 – Страница Profile.vue

6 Обработка ошибок и уведомления

6.1 Система уведомлений пользователя

Реализована глобальная система уведомлений с четырьмя типами сообщений: успех (зеленый), ошибка (красный), предупреждение (оранжевый) и информация (синий). Каждое уведомление отображается в правом верхнем углу экрана и автоматически скрывается через 3 секунды. Уведомления содержат понятный текст и, при необходимости, рекомендации по дальнейшим действиям.

6.2 Валидация и обработка ошибок при выполнении запросов

В приложении реализована многоуровневая система валидации и обработки ошибок, обеспечивающая стабильную работу и защиту от некорректных данных.

Категории обрабатываемых исключений

Клиентская валидация:

– проверка обязательных полей форм ввода (Рисунок 6.1);

– валидация формата email-адресов (Рисунок 6.2);

– контроль минимальной длины пароля (Рисунок 6.3);

– проверка числовых значений на соответствие диапазонам (Рисунок 6.4).

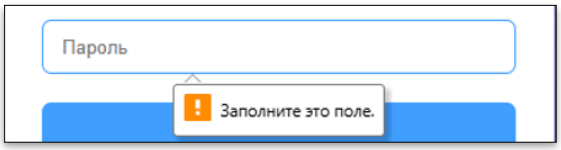

Рисунок 6.1 – Обработка ошибки пустого обязательного поля

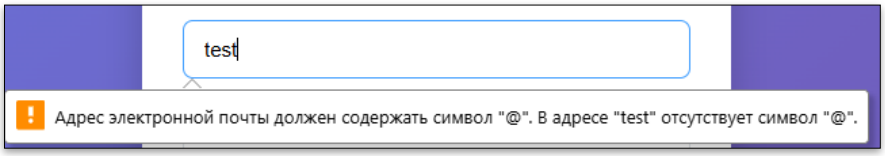

Рисунок 6.2 – Валидация формата email-адреса

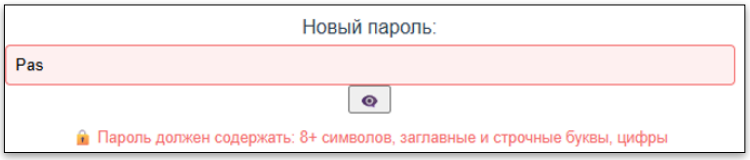

Рисунок 6.3 – Контроль минимальной длины пароля

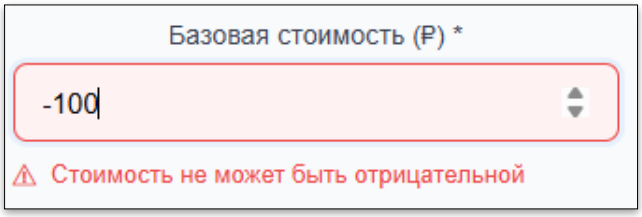

Рисунок 6.4 – Проверка значения на соответствие числовому диапазону

Серверная валидация:
– обработка ошибок аутентификации и авторизации (Рисунок 6.5);
– проверка доступности мест в расписаниях туров (Рисунок 6.6);
– валидация прав доступа к операциям (Рисунок 6.7).

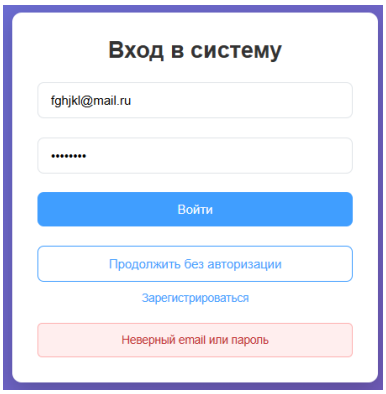

Рисунок 6.5 – Обработка ошибки авторизации

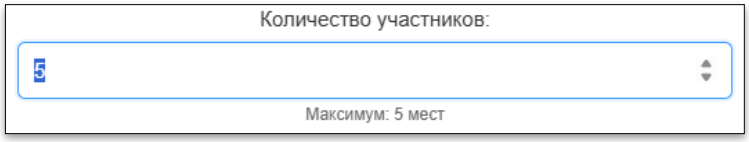

Рисунок 6.6 – Проверка доступности мест на бронирование

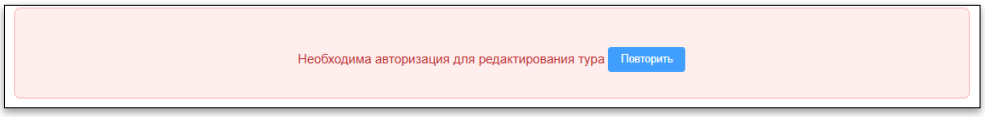

Рисунок 6.7 – Валидация прав доступа к операциям

Обработка сетевых ошибок:
– обработка случаев недоступности сервера (Рисунок 6.8).

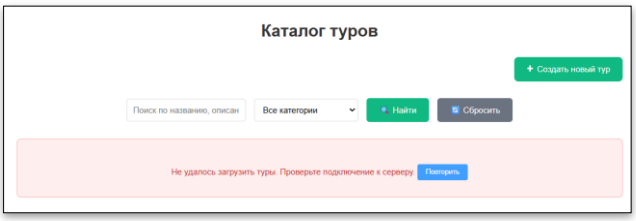

Рисунок 6.8 – Обработка случаев недоступности сервера

6.4 Подтверждение действий пользователя

Для необратимых операций (удаление тура, отмена бронирования) реализованы диалоговые окна подтверждения с четким описанием последствий действия и возможностью отмены. При отмене бронирования дополнительно запрашивается причина отмены.

7 Оптимизация производительности

7.1 Ленивая загрузка компонентов и маршрутов

Реализована динамическая загрузка компонентов с использованием defineAsyncComponent. Маршруты, не требующиеся при первоначальной загрузке (страница профиля, бронирований), загружаются асинхронно при переходе пользователя, что уменьшает размер основного бандла и ускоряет начальную загрузку.

7.2 Кэширование данных на клиенте

С использованием TanStack Query реализовано кэширование данных с временем жизни 5 минут. При повторном посещении страницы данные загружаются из кэша мгновенно, а в фоне выполняется проверка актуальности. При выполнении операций создания, обновления или удаления соответствующий кэш автоматически инвалидируется.

7.3 Оптимизация загрузки изображений

Для всех изображений используется атрибут loading="lazy", обеспечивающий загрузку только тех изображений, которые попадают в область видимости пользователя. Изображения оптимизированы по размеру и формату, для галерей реализована предзагрузка соседних изображений.

7.4 Пагинация для больших списков

Для каталога туров и списка бронирований реализована пагинация с загрузкой данных порциями по 10 элементов. При переключении страницы используется keepPreviousData для сохранения отображения предыдущих данных во время загрузки новых.